

AgentIA — Revisión del sistema, verificación y coste

El flujo de trabajo agéntico, la orquestación, lo que se mide y lo que cuesta

Ingeniería AgentIA

2026-09-29

Qué es este documento. Una revisión del sistema tal como se ejecuta hoy: el flujo de trabajo agéntico de generación (el pipeline LangGraph y su bucle de verificación), la orquestación que lo rodea, el diseño, los resultados de verificación obtenidos esta semana y un informe de coste/uso recogido de los propios almacenes de la plataforma — el sistema de registro local en SQLite y su espejo MLflow.

Está escrito *después* de que la plataforma produjera sus primeras compilaciones verificadas, por lo que separa lo medido de lo afirmado. Donde las dos fuentes de coste discrepan, este documento lo dice en lugar de elegir una en silencio.

Contenido

1	Resumen ejecutivo	2
2	El agente: qué hace y cómo está gobernado	2
2.1	La tarea, etapa por etapa	2
2.2	Autonomía con un tope duro	3
2.3	Dos motores, una decisión	3
2.4	Compuertas que posee el proyecto	3
2.5	El agente no puede calificar su propia tarea	3
2.6	Los prompts, la habilidad y el reflector	4
3	Orquestación	4
3.1	Dos puntos de entrada, un producto	4
3.2	Cola, ciclo de vida y streaming	4
3.3	Sustrato de medición y auto-mejora	4
4	Disponibilidad de la interfaz de usuario	5
5	Diseño	5
5.1	Inventario de servicios	5
6	Resultados de verificación	6
6.1	Escalado de cobertura: la palanca de fiabilidad	6
6.2	El verificador es intermitentemente inestable	6
7	Coste y uso	8
7.1	Sistema de registro (autoritativo)	8
7.2	Por tarea y por etapa (mediciones reales)	8
7.3	Espejo MLflow (y dónde se ha desviado)	9
7.4	Por qué la UI de MLflow puede verse vacía	9
8	Hallazgos y próximos pasos	9

1 Resumen ejecutivo

Cuatro hechos dominan esta revisión.

1. **La plataforma ya verifica su propia salida.** Hasta esta semana el sistema reportaba “0 de 15 sesiones verificadas”. La causa fue un defecto de montaje Docker de un solo carácter (un reetiquetado SELinux privado concatenado a una bandera de solo-lectura, que producía una opción de montaje inválida y hacía que toda compilación fallara antes de ejecutar Maven). Está corregido, y las primeras compilaciones verificadas están registradas.
2. **El flujo de trabajo agéntico es real y está gobernado.** Una máquina de estados LangGraph ejecuta ocho nodos — validador, cinco etapas de generación, sandbox, reparación — con una compuerta de cumplimiento determinista en cada etapa y un bucle de reparación acotado. Cada etapa de generación tiene dos implementaciones, MODELO y DETERMINISTA, despachadas una vez por sesión.
3. **La selección guiada por verificación es la palanca de fiabilidad medida.** Con una sola muestra el generador compila el 78% de las veces; muestrear tres candidatos y quedarse con el primero que compila eleva la cobertura a nivel de tarea al 100% a \$0.20 por servicio fiable.
4. **El verificador es intermitentemente inestable, y el espejo de coste se ha desviado.** El mismo espacio de trabajo congelado recompila 4 de 6 veces bajo carga y 8 de 8 veces secuencialmente; la causa principal es la misma configuración de montaje. Y el espejo MLflow subestima el sistema de registro en $\approx \$1.6$ (20%), de modo que el almacén local — no el espejo — es la cifra a citar.

2 El agente: qué hace y cómo está gobernado

En su núcleo, AgentIA es un único trabajador autónomo con una sola tarea: convertir un blueprint formal en un microservicio Spring Boot completo, compilable y con tests que pasan — y negarse a reportar éxito salvo que eso sea cierto. No es un chatbot que sugiere código; es un pipeline gobernado que escribe, revisa, compila y verifica, y se detiene cuando no puede. Esta sección describe a ese trabajador en términos sencillos: las etapas que ejecuta, las compuertas que lo restringen y el bucle que lo mantiene honesto.

2.1 La tarea, etapa por etapa

Ocho etapas llevan una especificación a un servicio verificado. Cada una tiene una única responsabilidad revisable, que es lo que hace auditable la salida.

Etapas	Produce	Por qué importa
Validador	aprueba o rechaza el blueprint	una mala especificación se ataja antes de gastar un token
Scaffolder	pom.xml, configuración, punto de entrada	la lista blanca de dependencias y el esqueleto, fijados por política
Dominio	entidades JPA + registros DTO inmutables	el modelo de datos, en un único lugar canónico
Servicio	repositorio, servicio, implementación	la lógica de negocio aislada en su propia capa
Controlador	endpoints REST + manejador global de errores	una superficie de API uniforme
Test	tests unitarios derivados de los escenarios de aceptación	todo comportamiento declarado tiene un test
Sandbox	compila y ejecuta los tests, offline	prueba, no promesa

Reparación	un parche quirúrgico, luego re- ejecuta	autocorrección, acotada
------------	--	-------------------------

El orden no es arbitrario: cada etapa construye solo sobre los artefactos que las anteriores ya produjeron, de modo que una decisión tomada aguas arriba — el nombre del paquete, una entidad, un contrato — nunca se contradice silenciosamente aguas abajo.

2.2 Autonomía con un tope duro

El pipeline es deliberadamente un bucle acotado, mostrado en la Figura 2. La generación corre en una dirección; luego el sandbox compila. Si la compilación falla, la etapa de reparación lee el error real de Maven, aplica un parche estrecho y vuelve a compilar — hasta un límite fijo (tres intentos según la constitución, cinco en un archivo de configuración, una inconsistencia señalada en la Sección 8). Al llegar al límite, la sesión se marca **BLOQUEADA** y se entrega a un humano.

Ese tope es el punto, no una limitación: el agente puede corregirse, pero no puede iterar indefinidamente, y no puede devolver en silencio algo que no pudo compilar. La autonomía acotada es la propiedad que un directorio puede avalar — el coste de una respuesta equivocada está limitado a una revisión, nunca a un gasto de tokens sin freno ni a un artefacto no verificable.

2.3 Dos motores, una decisión

Cada etapa de generación tiene dos implementaciones: **DETERMINISTA** (plantillas offline, cero llamadas al modelo, salida idéntica byte a byte) y **MODELO** (un LLM de uso general, actualmente **deepseek-flash**, conducido por un conjunto de instrucciones versionado). La elección se hace una vez por sesión y se registra, nunca se re-decide a medias, de modo que una sesión no puede mezclar ambas. El motor determinista existe por coste y por auditoría: la misma entrada debe producir la misma salida cuando un auditor lo pregunta, y una sesión debe poder ejecutarse sin red y sin clave de API.

2.4 Compuertas que posee el proyecto

Antes de escribir cualquier archivo, pasa una compuerta de cumplimiento que fusiona dos familias de reglas — el análisis constitucional de tests y el escaneo de arquitectura de seguridad. Cada hallazgo lleva una atribución: **LOCAL** (esta etapa posee el archivo infractor) o **ACUMULADO** (una preocupación de todo el proyecto, p. ej. “los errores deben centralizarse”). Una violación local rechaza el candidato y gasta un intento de corrección; una acumulada se registra pero nunca se le culpa a una sola etapa.

La propiedad clave para el directorio: **estas son las reglas del propio proyecto, aplicadas de forma determinista — no la opinión del modelo sobre su propia salida.**

2.5 El agente no puede calificar su propia tarea

La decisión de diseño determinante es que el test de aceptación *no* lo escribe el modelo. Tras cada etapa, la plataforma inyecta su propio test de contrato de persistencia — un arranque de contexto Spring con `ddl-auto=validate` contra el `schema.sql` generado — y luego compila y ejecuta todo dentro de un contenedor Docker sellado, sin red y con una caché de dependencias de solo lectura. El modelo nunca ve este test, por lo que no puede escribir para él.

Esto importa porque los tests unitarios del propio modelo simulan la capa de persistencia: un servicio generado cuyo esquema contradice sus entidades pasaría todos los tests escritos por el modelo y fallaría el de la plataforma. Esa clase de defecto se ha capturado aquí, y es exactamente la brecha que este diseño cierra. Los conteos de tests se leen de la propia salida de la compilación, nunca se fabrican, y una compilación que reporta éxito sin ejecutar ningún test se registra como no verificada.

2.6 Los prompts, la habilidad y el reflector

Tres piezas de texto autorado conducen al modelo, y merece nombrarlas porque son el “programa” real y revisable de la plataforma:

- **Los cinco prompts de etapa** (`resources/instructions/*.md`). Uno por etapa, cada uno en cuatro secciones fijas — *Contrato tecnológico*, *Reglas*, *Contrato de salida*, *Prohibiciones*. Se inyectan textualmente antes de cada petición al modelo y se versionan por contenido, de modo que un artefacto registra exactamente qué texto de instrucción lo produjo.
- **La habilidad** (`resources/skills/layer_architecture.md`). Un único documento que codifica la disciplina de capas: una clase por capa; depender solo de la capa inmediatamente inferior; nunca importar un repositorio desde un controlador. El slot de habilidad actualmente no apunta a nada — dos archivos de habilidad más están vacíos y el puntero activo no existe — así que la habilidad está definida pero aún no conectada.
- **El prompt del reflector** (`scripts/skillopt/prompts/analyst_error.md`, adaptado de Skillopt, Apéndice C.2.1). Es el texto que convertiría diagnósticos registrados en ediciones de habilidad: consume el histograma de reglas, la atribución por etapa, el código de salida de la compilación, el estado terminal y la bandera de fallback de verificación — el canal de diagnóstico, no un veredicto binario.

3 Orquestación

3.1 Dos puntos de entrada, un producto

El sistema expone dos formas de iniciar una sesión, y esta revisión confirma que ambas ahora llegan al modelo y ambas verifican el espacio de trabajo:

- **Ruta de grafo** — `POST /sessions` encola una sesión cuyo worker ejecuta el grafo compilado de extremo a extremo, con las credenciales pasadas por la cabecera `X-LLM-API-Key`.
- **Ruta secuencial** — `POST /sessions/quick-start` con `autoRun`, y `/orchestrator/pipeline/run`, conducen `pipeline_runner`, que ejecuta las mismas etapas de generación, la misma verificación hermética, una auditoría de seguridad estática y las fases de DevOps/exportación.

La ruta secuencial antes generaba código y *nunca lo compilaba*; ahora ejecuta la misma veta de verificación que el grafo, de modo que las dos rutas no pueden divergir.

3.2 Cola, ciclo de vida y streaming

Una cola FIFO (`queue_service.py`) limita la concurrencia a dos ejecuciones de sandbox simultáneas. El ciclo de vida (`lifecycle_service.py`) recorre `REQUISITOS` → `ARQUITECTURA` → `MODELO_DATOS` → `CÓDIGO_TESTS` → `AUDITORÍA_SEGURIDAD` → `DEVOPS` → `EXPORTACIÓN` → `COMPLETADO`, con pausa, reanudación y cancelación en vuelo y streaming en vivo por Server-Sent Events a la pantalla de monitor.

3.3 Sustrato de medición y auto-mejora

Dos capas más acompañan a la generación. El instrumento de diagnóstico (`conformance_diagnostics.py`) es un puntuador determinista y atribuido por regla que persiste un registro duradero por sesión y produce un informe de corpus. El bucle de optimización de habilidades (features 014–015) está *construido* — recopilar, reflexionar, aplicar, una compuerta de mejora estricta, contribución por habilidad por leave-one-out y desalojo con suelo de evidencia — pero deliberadamente no se ejecuta, por las razones expuestas en la Sección 6.

4 Disponibilidad de la interfaz de usuario

La superficie de producto es una aplicación de página única React/Vite (la antigua app Streamlit se conserva como `frontend-legacy/`). Se organiza como un espacio de trabajo de diez pestañas con etiquetas en español, tabulado abajo; un stepper de ciclo de vida refleja la máquina de fases del backend, y los diagramas de arquitectura y entidad-relación se renderizan a través de un visor Mermaid.

Pestaña	Qué expone
0. Resumen	panel ejecutivo; inicio rápido 1-clic; modo Auto-Pilot vs paso a paso
1. Requisitos	historias de usuario BDD (Given/When/Then) y exportación <code>spec.md</code>
2. Arquitectura	arquitectura en 4 capas, catálogo de componentes, contratos REST (Mermaid)
3. Modelos & SQL	entidades JPA, registros DTO, <code>schema.sql</code> , diagrama ER
4. Blueprints	ingesta drag-and-drop de <code>spec.md</code> / JSON blueprint y validación
5. Monitor Live	logs en vivo por Server-Sent Events, progreso de fases, pausa/re-anudar/cancelar
6. Código & Fix	explorador de código, tests, diffs de reparación, intervención manual en sesiones bloqueadas
7. Calidad SAST	auditoría estática de secretos/CVE/arquitectura, compuerta de calidad, remediación 1-clic
8. DevOps & Demo	Dockerfile, compose, CI/CD, manifiestos k8s, despliegue local, smoke test, consola REST
9. Entrega Git	descarga ZIP y publicación Git atómica

Más allá de las pestañas: un panel de configuración LLM (proveedor: Gemini, Groq, OpenAI, DeepSeek o mock offline; clave API; modelo; verificación de conexión), autenticación en la pantalla de login, un conmutador de tema claro/oscuro y una lista de sesiones con estado. La interfaz refleja la verificación honesta de tres estados del backend (verificado / verificado con hallazgos / no evaluable), tras una auditoría de frontend que eliminó varios lugares donde la interfaz reportaba resultados que no habían ocurrido (un veredicto “UP” de smoke test fabricado, una consola REST que inventaba respuestas 200/201/204 y un formulario CRUD que inventaba un registro creado).

5 Diseño

5.1 Inventario de servicios

Servicio	Rol
<code>requirements_service</code>	descomposición de historias BDD y síntesis de spec
<code>architecture_service</code>	arquitectura en 4 capas y catálogo de componentes
<code>model_sql_service</code>	entidades JPA, DTOs, SQL de esquema/datos, diagrama ER
<code>test_analysis_service</code>	analizador estático de reglas constitucionales (familia A)
<code>security_service</code>	SAST, secretos, conformidad de arquitectura (familia B)
<code>conformance_diagnostics</code>	puntuador de conformidad determinista y atribuido por regla
<code>corpus_report</code>	informe de medición por lote/corpus

<code>workspace_verification</code>	eliminar VCS, inyectar test de contrato, ejecutar la compilación
<code>verification_selection</code>	best-of- k : generar, verificar, detenerse en el primer éxito
<code>platform_verification</code>	test de contrato de persistencia escrito por la plataforma
<code>skill_contribution</code>	contribución por habilidad por leave-one-out
<code>llm_factory</code>	detección de proveedor y construcción del cliente del modelo
<code>structured_output</code>	salida del modelo restringida por esquema
<code>injection_guard</code>	defensa contra prompt-injection del reflector
<code>queue_service /</code> <code>lifecycle_service</code>	conurrencia y orquestación de fases
<code>docker_service / devops /</code> <code>export / git</code>	despliegue en contenedor, CI, ZIP, publicación
<code>registro de coste + espejo</code> <code>mlflow</code>	coste por llamada y por sesión, espejo

6 Resultados de verificación

6.1 Escalado de cobertura: la palanca de fiabilidad

La selección guiada por verificación — generar k candidatos, verificar cada uno, quedarse con el primero que pasa — se mide aquí, no se toma prestada de otra clase de tarea. El brazo de control del experimento de esta semana son 6 tareas, cada una remuestreada 3–4 veces con las mismas instrucciones y modelo; la fórmula de cobertura es $\text{coverage}(k) = 1 - (1 - p)^k$ (*Large Language Monkeys*, arXiv:2407.21787).

Métrica	Valor
pass@1 (una muestra)	78.3%
cobertura $k=1$	83.3%
cobertura $k=2$	83.3%
cobertura $k=3$	100.0% 6 de 6 tareas
cobertura $k=4$	100.0%

A \$0.065 por servicio generado, tres candidatos cuestan **\$0.20 por servicio fiable**. Advertencias, declaradas y no suavizadas: $n=6$ tareas, un modelo, un proveedor; esto es una demostración de la palanca, no un benchmark, y la dirección (la cobertura es monótona en k) es robusta mientras que el 100% es “6 de 6 tareas”.

6.2 El verificador es intermitentemente inestable

El mismo espacio de trabajo congelado, recompilado bajo carga antes, pasó 4 de 6 veces; recompilado secuencialmente, dos espacios de trabajo pasaron 8 de 8 cada uno. El fallo, cuando ocurre, es una `PluginContainerException` de surefire sobre un import foráneo de `surefire-shared-utils` — después de que todos los tests hayan pasado.

La inestabilidad no es, por tanto, un defecto de código por espacio de trabajo. Es intermitente y correlacionada con la carga, y la hipótesis principal es la configuración de montaje: `DOCKER_MOUNT_SUFFIX=:Z` es un reetiquetado SELinux *privado*, correcto para un espacio de trabajo por sesión pero incorrecto para una *caché Maven compartida de solo lectura* montada por muchos contenedores concurrentes. Dos contenedores reetiquetando una fuente compartida con `:Z` es una causa conocida de exactamente esta clase de fallo intermitente de realm de clases. La corrección propuesta es aplicar `:Z`

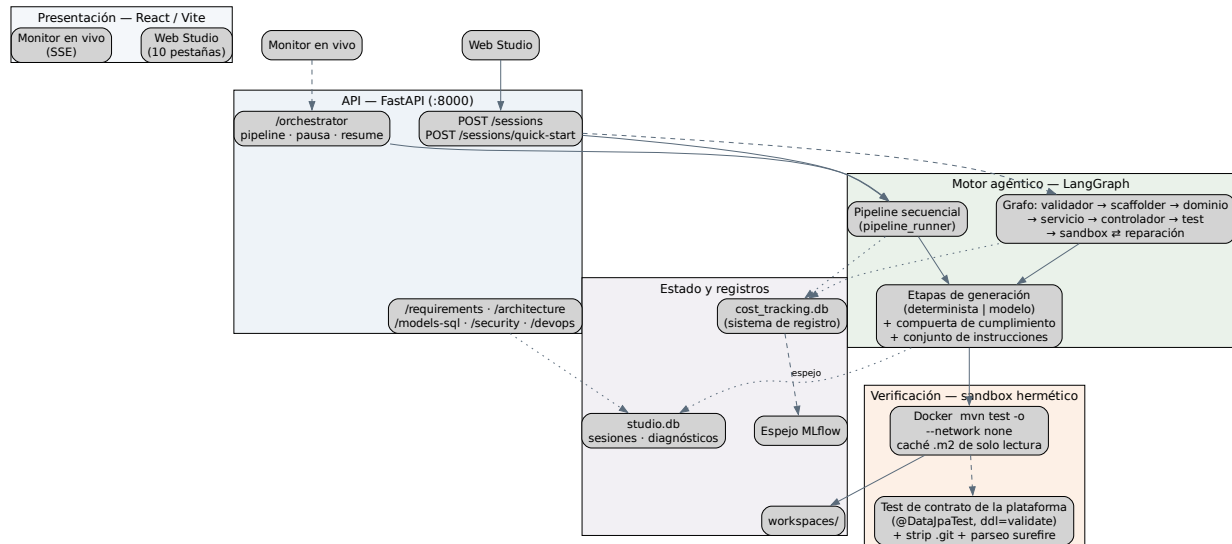


Figura 1: Arquitectura del sistema: el estudio React sobre la API FastAPI, el motor agéntico LangGraph, el sandbox de verificación hermético y los almacenes de estado/coste.

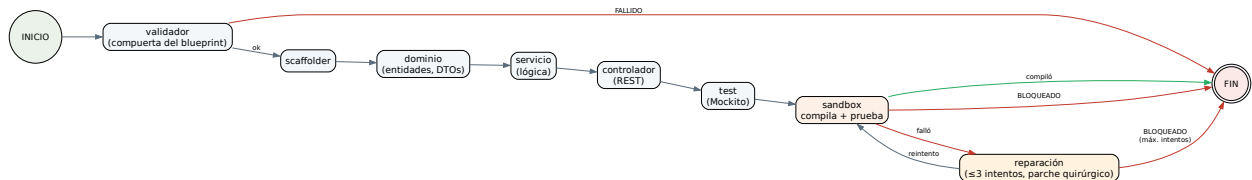


Figura 2: La máquina de estados de generación: una columna lineal con un ciclo acotado de reparación-sandbox.

solo al espacio de trabajo y dejar la caché sin reetiquetar (o :z), confirmada por una compuerta de repetibilidad.

Por qué esto condiciona todo lo posterior. Un verificador que se voltea $\approx 33\%$ de las ejecuciones no puede servir como oráculo de admisión: la cobertura prometería de más, una compuerta de admisión promovería ruido, y el teorema de Ratchet ([arXiv:2605.22148](#)) dice que un juez tan ruidoso no retira nada a ningún tamaño de muestra. Hacer el oráculo determinista es, por tanto, el prerequisite de toda medición posterior — incluido el bucle de optimización de habilidades, que permanece cerrado tras él.

7 Coste y uso

Los costes se registran por llamada y por sesión en el almacén SQLite local (`backend/cost_tracking.db`), que es el **sistema de registro**, y se espejan en MLflow (`mlflow.db`). Ambos se comparan aquí, no se fusionan.

7.1 Sistema de registro (autoritativo)

Alcance	Llamadas	Coste (USD)	Tokens salida
Experimento de polaridad (p4)	750	6.58	—
Línea base cinco congelados (corpus)	120	0.63	—
Diez difíciles	50	0.64	—
Sondas / sesiones sueltas	≈ 250	≈ 0.16	—
Total	1172	8.01	12.28M

Las 1172 llamadas fueron `deepseek-flash`. Total de tokens de entrada 6.45M, de salida 12.28M. El coste dominante es el experimento de polaridad de instrucciones, que es una medición terminada (un nulo), no un drenaje continuo.

7.2 Por tarea y por etapa (mediciones reales)

En el corpus de diez servicios “difíciles”, un servicio generado cuesta **\$0.052–\$0.077**, para un total de corpus de **\$0.6357** sobre 50 llamadas — aproximadamente **\$0.005 por llamada al modelo**, y el coste escala con la complejidad del blueprint (tokens de entrada y salida), no con los reintentos.

Etapas (un servicio)	Tok entr.	Tok sal.	Coste
Scaffolder	2,200	6,620	\$0.0040
Dominio	2,669	1,759	\$0.0012
Servicio	3,250	5,166	\$0.0034
Controlador	3,925	4,210	\$0.0029
Test	5,711	9,108	\$0.0061
Total	17,755	26,863	\$0.0176

La etapa de test es la más cara porque su salida es la mayor; las de scaffolder y dominio son las más baratas. Ninguna etapa domina, así que el coste es predecible y lineal en el tamaño del servicio.

7.3 Espejo MLflow (y dónde se ha desviado)

Experimento	Runs	Coste (USD)	Tok entr.	Tok sal.
Default (legado)	877	5.41	4.63M	8.20M
agentia (actual)	114	0.98	0.78M	1.52M
Total espejo	991	6.39	5.40M	9.72M

El espejo se ha desviado. MLflow totaliza **\$6.39** frente a los **\$8.01** del sistema de registro — una subestimación de $\approx \$1.62$ (20%), con ≈ 181 llamadas menos y totales de tokens que también discrepan. El espejo está dividido entre un experimento legado y uno actual, y solo se escriben métricas por llamada (faltan los agregados de sesión). La afirmación anterior de que “el espejo funciona y coincide” ya no es cierta para los datos actuales. Hasta que se repare el sink, el almacén local es la única cifra a citar, y un informe de coste generado solo desde MLflow subestimaría el gasto.

7.4 Por qué la UI de MLflow puede verse vacía

Dos causas independientes. Primera, los datos viven en `mlflow.db` (SQLite); el almacén de archivos `mlruns/` — que `mlflow ui` abre *por defecto* — está vacío, de modo que un `mlflow ui` a secas no muestra nada. El servidor debe arrancarse con `-backend-store-uri sqlite:///mlflow.db`. Segunda, incluso en el almacén correcto los registros están en el experimento equivocado: los 168 agregados de sesión y 623 de las ejecuciones por llamada están en el experimento legado `Default`, mientras que el experimento `agentia` (que nombra la configuración) guarda 0 registros de sesión y 119 por llamada. El targeting de experimento del sink no alcanza a los registros de sesión.

8 Hallazgos y próximos pasos

1. **Hacer determinista el verificador antes de gastar más.** Separar el sufijo de montaje (espacio de trabajo :Z, caché compartida sin reetiquetar) y añadir una compuerta de repetibilidad: un espacio de trabajo congelado debe recompilar N/N veces.
2. **Hacer que la medida pueda moverse.** El instrumento de conformidad lee 0 en compilaciones que fallan; exponer las clases de fallo del analizador de reparación como hallazgos acumulados para que un fallo de compilación sea atribuible.
3. **Usar la selección guiada por verificación como mecanismo de fiabilidad ya.** Está medida (\$0.20/servicio a $k=3$), no necesita entrenamiento y es la palanca que la literatura sitúa en primer lugar.
4. **Reparar el espejo de coste.** El espejado a nivel de sesión no se está poblando; restaurarlo o la UI de MLflow subestima el gasto.
5. **Resolver la inconsistencia del límite de reparación** (constitución 3 vs `config.py` 5).
6. **Dejar cerrado el bucle de optimización de habilidades** hasta que los puntos 1 y 2 pasen sus compuertas; el experimento de polaridad de instrucciones que era su entrada más prometedora es un nulo (prohibición vs control: 1 victoria, 3 derrotas, 2 empates).